

Рахмаева Светлана Фарисовна

QA Engineer (Manual)

Тестировщик с опытом полного цикла QA: от анализа требований на ранних стадиях до релиза. Работала с веб-продуктами и мобильными приложениями (iOS/Android).

Выстраивала тестовый процесс с нуля — в том числе на проектах стадии MVP: выбирала инструментарий и формировала документацию с нуля. Проводила приёмочное тестирование продуктов от внешних подрядчиков, оценивая соответствие требованиям и готовность к передаче.

В работе тесно взаимодействовала с аналитиками, разработчиками и менеджерами — участвовала в обсуждении требований, помогала находить узкие места до начала разработки.



Виды тестирования

- Функциональное, регрессионное, смоук
- UI/UX и исследовательское
- API-тестирование
- Нагрузочное тестирование
- Кросс-платформенное
- End-to-end
- Базы данных (SQL - запросы)

Ключевые достижения

- ✓ **Внедрила нагрузочное тестирование**
Самостоятельно, с нуля, без внешней поддержки
- ✓ **Создала базу тест-кейсов в Zephyr**
Тестовая документация выстроена с нуля
- ✓ **Написала внутренние инструкции по API**
Для быстрой адаптация новых сотрудников
- ✓ **Снизила количество критических багов**
Системный подход к тестированию на каждом этапе.
- ✓ **Улучшила UX на уровне интерфейса**
Выявила и подсветила проблемы с placeholder и валидацией полей

Инструменты и технологии: Postman, Swagger, Charles, Devtools, Android Studio, Apache Kafka, RabbitMQ, DBiever, PostgreSQL, Docker, Git, JMeter, influxdb, Zephyr, Jira, Confluence.

Подход к тестированию

Тестовое покрытие выстроено по принципу Shift Left: я подключаюсь на этапе требований, а не после разработки. Это позволяет выявлять дефекты раньше и снижать стоимость их исправления. Ниже — что именно покрыто тестированием в рамках проекта.

- **UI-тестирование**

Проверка отображения, поведения элементов, состояний формы (пустое, заполненное, ошибка)

- **Валидация данных**

Граничные значения, пустые поля, спецсимволы, форматы телефона и email

- **Cross-browser тестирование**

Chrome, Firefox, Safari — поведение формы и модального окна

- **Регрессионное тестирование**

Проверка, что новый функционал не ломает уже работающее

- **API-тестирование**

GET / POST / PUT / DELETE, статус-коды, схемы ответов, авторизация

- **Негативные сценарии**

Невалидные данные, отсутствие токена, недоступность сервиса

- **Мобильное тестирование**

iOS (iPhone 13) и Android — адаптивность, нативные жесты

- **Нагрузочное тестирование**

JMeter, 3 сценария: 50 / 100 / 150 RPS

Анализ рисков

Перед каждым релизом я анализирую, что именно может сломаться, и расставляю приоритеты тестирования. Это позволяет не тратить время на низкорисковые сценарии и сфокусироваться на том, что влияет на пользователя.

- **Потеря заявки при сетевом сбое**

Если соединение прерывается в момент отправки — заявка не создаётся, но пользователь об этом не узнаёт.

- **Нестабильность UI при слабом соединении**

Форма зависает без индикатора загрузки — пользователь не понимает, что происходит

- **Дублирование заявок**

Повторное нажатие кнопки «Заказать» без защиты на фронте создаёт несколько записей в базе.

- **Ошибки обработки API**

Некорректные коды ответа (500, 422) не обрабатываются на фронте — пользователь видит пустой экран.

- ❏ Основной риск — потеря пользовательской заявки при сетевых сбоях или повторном нажатии кнопки «Заказать». Приоритет тестирования был сфокусирован на: корректности отправки данных, обработке ошибок API, стабильности UI при медленном соединении и защите от `duplicate request`.

Документация

Тест-кейс:

ТС-001 — Успешная заявка на эвакуатор с валидными данными

Предусловия: Пользователь авторизован, интернет-соединение стабильно

Приоритет: Высокий

Шаг	Действие	Ожидаемый результат
1	Нажать кнопку «Заказать услугу» на главном экране	Появляется выпадающий список доступных услуг
2	Выбрать «Вызов эвакуатора» в выпадающем списке	Открывается форма с полями: Имя, Телефон, Email
3	Ввести корректное имя (например, «Анна Иванова»)	Имя введено корректно
4	Ввести корректный номер телефона (например, «+7 900 123-45-67»)	Номер телефона введён корректно
5	Ввести корректный email (например, « test@mail.ru »)	Email введён корректно
6	Нажать кнопку «Заказать»	Появляется модальное окно: «Мы получили вашу заявку. Ожидайте звонка»

Ожидаемый результат: Заявка успешно отправлена, отображается модальное окно с подтверждением

Статус: Пройден / Не пройден

Баг-репорт:

ID: BUG-001

Название: Модальное окно не появляется после нажатия кнопки «Заказать»

Связанный тест-кейс: TC-001

Приоритет: Высокий

Серьёзность: Критическая

Окружение:

- Устройство: iPhone 13
- ОС: iOS 16.5
- Версия приложения: 2.1.0

Предусловия: Пользователь авторизован, интернет-соединение стабильно

Шаги для воспроизведения:

1. Нажать кнопку «Заказать услугу» на главном экране
2. Выбрать «Вызов эвакуатора» в выпадающем списке
3. Заполнить поля: Имя, Телефон, Email валидными данными
4. Нажать кнопку «Заказать»

Фактический результат: Модальное окно с подтверждением не появляется. Экран остаётся без изменений

Ожидаемый результат: Появляется модальное окно с текстом: «Мы получили вашу заявку. Ожидайте звонка»

SQL — работа с базой данных

При тестировании я верифицирую данные напрямую через базу — это позволяет убедиться, что заявка действительно создана, статус изменился корректно, а дубликатов нет. Ниже — примеры запросов, которые я использую в работе.

-- Проверяем, что заявка создана и в статусе 'pending'

```
SELECT id, user_id, status, created_at  
FROM orders  
WHERE status = 'pending'  
ORDER BY created_at DESC  
LIMIT 10;
```

-- Ищем дублирующие заявки за последние 5 минут

```
SELECT user_id, COUNT(*) AS cnt  
FROM orders  
WHERE created_at > NOW() - INTERVAL '5 minutes'  
GROUP BY user_id  
HAVING COUNT(*) > 1;
```

-- Смотрим распределение статусов после нагрузочного теста

```
SELECT status, COUNT(*) AS total  
FROM orders  
GROUP BY status;
```

Тестирование API через Postman

Я тестирую API не только как «отправила запрос — получила ответ», а проверяю полный цикл: корректность статус-кодов, схему ответа, поведение при невалидных данных и отсутствии токена авторизации. Коллекции храню в Postman с разделением по окружениям (dev / staging / prod).

GET /api/services	→200 OK, schema валидна, поле name не пустое
POST /api/orders (валидные данные)	→ 201 Created, id присутствует в теле, латентность < 500ms
POST /api/orders (невалидные данные)	→ 422 Unprocessable Entity, поле errors[] содержит описание
POST /api/orders (без авторизации)	→ 401 Unauthorized, без Bearer token запрос отклонён
PUT /api/orders/:id	→ 200 OK, поле updated_at изменилось
DELETE /api/orders/:id	→ 204 No Content, запись удалена в базе

```
{
  "id": 1042,
  "status": "created",
  "service": "evacuator",
  "user": {
    "name": "Anna Ivanova",
    "phone": "+79001234567"
  },
  "created_at": "2025-05-12T10:42:00Z",
  "confirmed": true
}
```

Performance — результаты нагрузочного тестирования

Я самостоятельно настроила сценарии нагрузочного тестирования в JMeter — с нуля, без внешней поддержки. Тестировала endpoint /create-order при трёх уровнях нагрузки: нормальном, повышенном и пиковом. По результатам выявила узкое место — пул соединений с базой данных.

Нагрузка	Avg response time	Error rate	Статус
50 RPS	220ms	0%	✔ Норма
100 RPS	680ms	0.2%	⚠ Допустимо
150 RPS	1800ms	3.1%	❌ Критично

❏ При нагрузке 150 RPS время ответа endpoint /create-order увеличивалось с 220ms до 1.8s. Зафиксированы intermittent 502 errors при пиковых значениях. Рекомендации по итогам: оптимизировать пул соединений с БД, добавить кэширование справочника услуг, установить rate limiting на стороне API.

Измеримые результаты

Всё, что сделано — измеримо. Ниже — ключевые цифры по проекту. Тест-кейсы, баг-репорты и коллекции созданы с нуля, в том числе нагрузочное тестирование.

400+

тест-кейсов в Zephyr

180+

баг-репортов в Jira

15+

API-коллекций в
Postman

3

сценария
нагрузочного
тестирования

□ Каждая цифра получена в рамках одного проекта — с нуля до выстроенного QA-процесса.

AI, используемые мной в работе

- **Claude** — генерация и оптимизация автотестов в Postman: pre-request скрипты, pm.test() проверки, покрытие негативных сценариев
- **ChatGPT** — помощь в написании тест-кейсов и генерация тестовых данных
- **Perplexity** — быстрый технический ресёрч: стандарты API, best practices, документация
- **Qwen** — структурирование и оформление отчётов по тестированию

Примеры промтов:

Claude

Напиши pm.test() скрипты для POST /api/orders.
Проверь: статус-код 201, наличие поля id в теле ответа, латентность < 500ms, и что поле status равно "created"

ChatGPT

Проверь мои тест-кейсы и дай рекомендации по улучшению. Оцени: достаточно ли покрыты негативные сценарии, есть ли граничные значения, понятны ли шаги для воспроизведения, нет ли дублирования. Укажи конкретно, что доработать и что добавить.

Perplexity

Какие граничные сценарии чаще всего упускают QA-инженеры при тестировании идемпотентности PUT и DELETE запросов в REST API

Qwen

Составь отчёт по результатам тестирования на основе данных: 50 тест-кейсов, 7 баг-репортов. Критические баги: заявка создается, но не отображается в админ-панели. Структура: резюме, что проверялось, найденные проблемы, рекомендации

- ☐ Использую нейросети как рабочий инструмент — не для замены экспертизы, а для ускорения рутины и повышения качества.

Для связи со мной:

Тел.: 8 (987) 067 0000

ТГ: SvetikLanik

Мое портфолио:

<https://github.com/SvetlanaRakhmaeva/qa-portfolio>

✓ Я выявляю риски раньше, чем они становятся проблемами.

✓ Я создаю процессы, которые держат качество под контролем.

✓ Я превращаю сложные сценарии в управляемые и безопасные.

